

M09 MATERI 1

1. TUJUAN

CPMK : Mahasiswa dapat memahami dan mengimplementasikan ASP .Net Core.

Sub-CPMK :

- a. Mahasiswa dapat memahami dan mengimplementasikan ASP .Net Core..

2. DURASI WAKTU

2 pertemuan x 4 jam

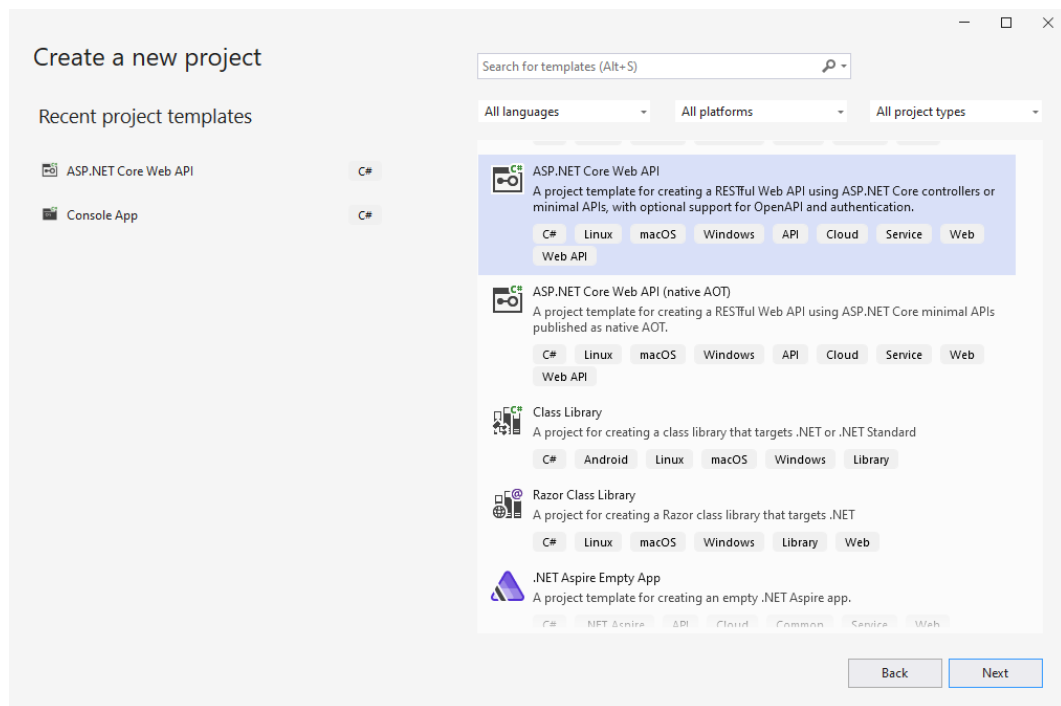
3. DASAR TEORI

ASP .NET Core

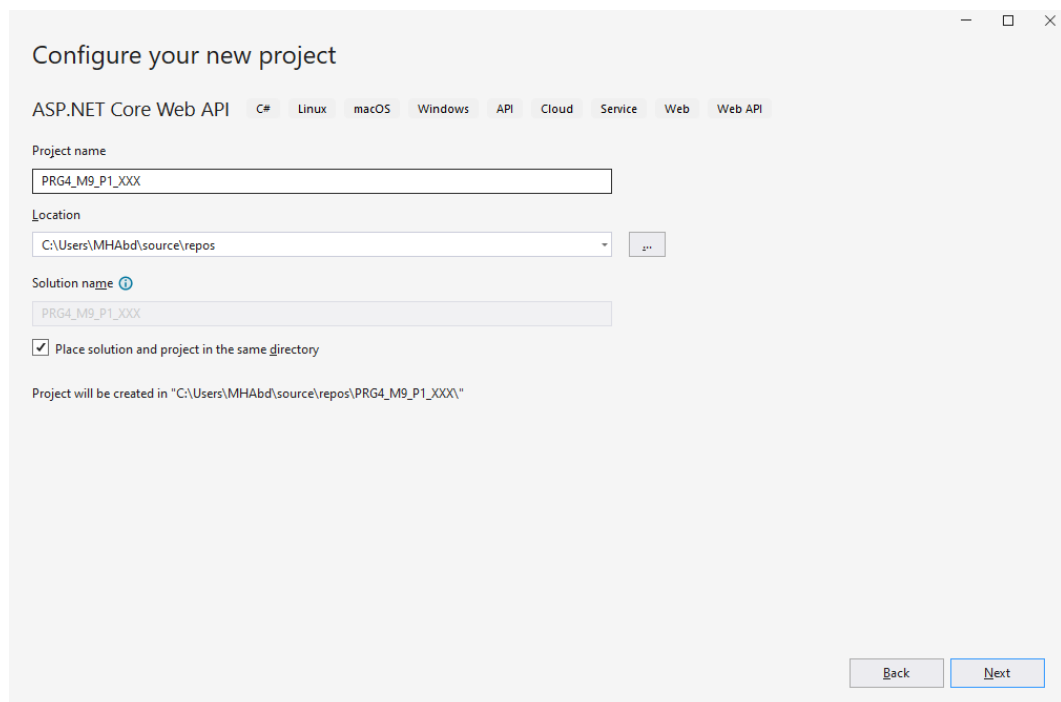
4. Percobaan

• Membuat Projek ASP .NET Core API Sederhana

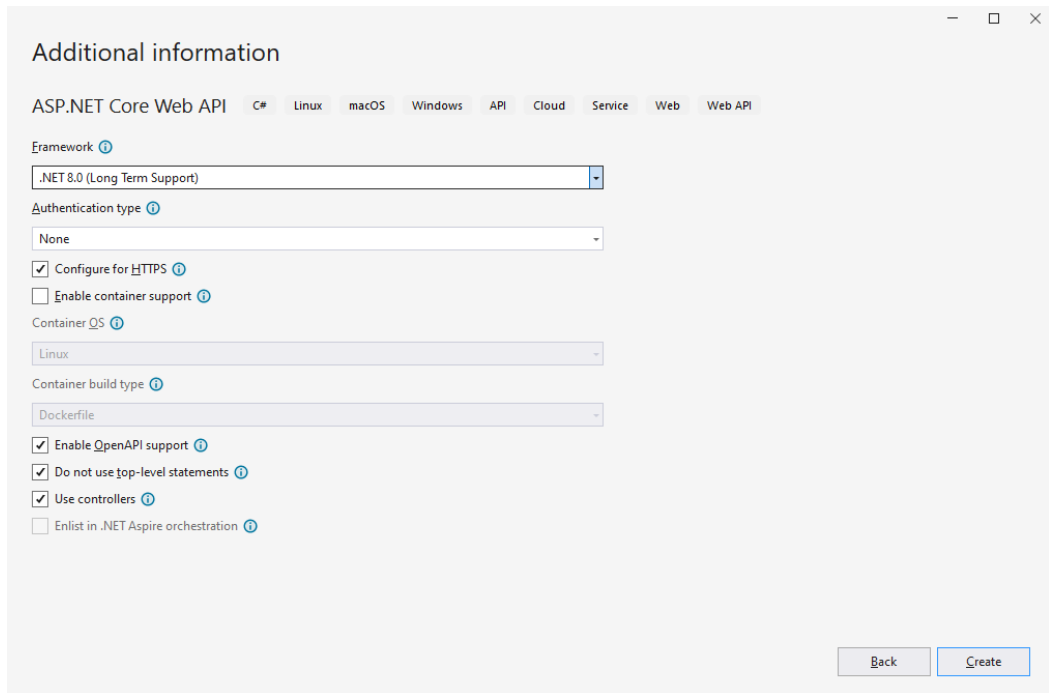
1. Buatlah projek baru dalam Visual Studi 2022 dengan cara klik **Create a New Project**, lalu pilih **ASP.Net Core Web API**



2. Berikan nama projek baru ini dengan format **PRG4_M9_P2_XXX** (XXX adalah 3 angka paling belakang di NIM anda), kemudian klik Next.



3. Pilihlah Framework .NET 8.0, kemudian klik create



Additional information

ASP.NET Core Web API C# Linux macOS Windows API Cloud Service Web Web API

Framework 

.NET 8.0 (Long Term Support)

Authentication type 

None

☒ Configure for HTTPS 

☐ Enable container support 

Container OS 

Linux

Container build type 

Dockerfile

☒ Enable OpenAPI support 

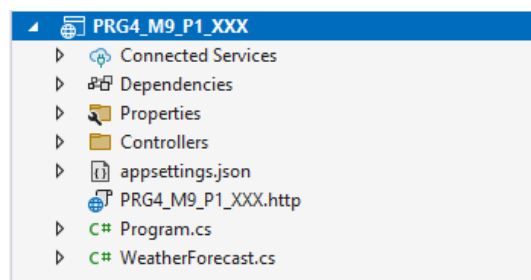
☒ Do not use top-level statements 

☒ Use controllers 

☐ Enlist in .NET Aspire orchestration 

Back Create

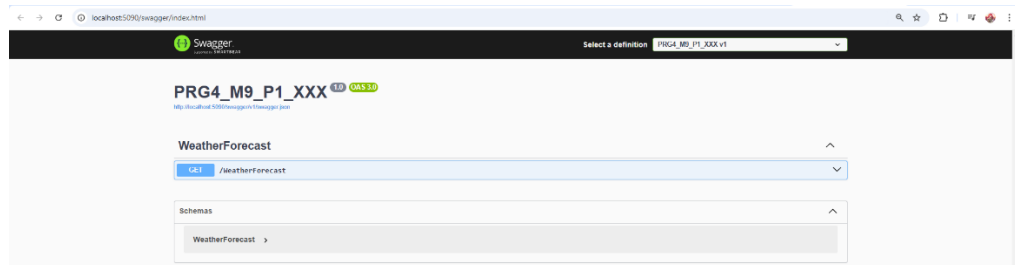
4. Setelah proyek dibuat, lihatlah struktur folder proyek yang tampil disamping.



- **Connected Services:** Folder ini digunakan untuk menambahkan dan mengelola layanan yang dibutuhkan oleh aplikasi, seperti RESTful services, atau layanan Azure.
- **Dependencies:** Berisi semua pustaka atau paket eksternal yang digunakan proyek Anda, seperti paket NuGet atau framework tertentu.
- **Properties:** Folder ini berisi metadata untuk proyek, seperti file konfigurasi launchSettings.json, yang berisi pengaturan untuk menjalankan dan debugging aplikasi.
- **Controllers:** Folder ini berisi controller, yang menangani permintaan HTTP masuk, memprosesnya, dan mengembalikan respons yang sesuai.
- **appsettings.json:** File konfigurasi di mana Anda menyimpan pengaturan aplikasi seperti connection string, API key, atau pengaturan untuk lingkungan (development, production, dll.).
- **PRG4_M9_P2_XXX.http:** File ini kemungkinan adalah file untuk pengujian, di mana Anda bisa menuliskan permintaan HTTP ke aplikasi dan mengujinya langsung.

- **Program.cs:** Titik masuk (entry point) untuk aplikasi. Di sini, aplikasi dan web host dikonfigurasi dan dijalankan.
- **WeatherForecast.cs:** Ini adalah kelas model yang sering disertakan dalam proyek template sebagai contoh.

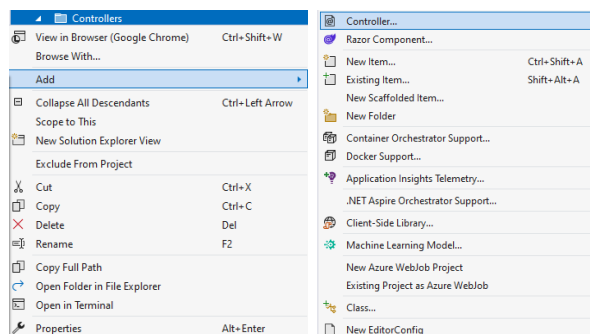
5. Selanjutnya jalankan proyek, kemudian akan tampil seperti ini!



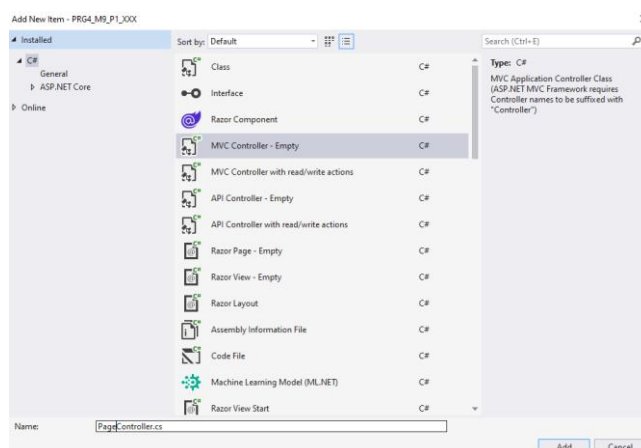
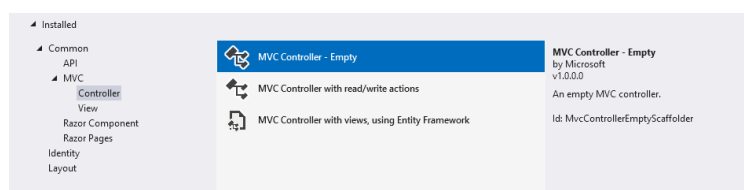
6. Cobalah setiap endpoint dari api yang ada!

• Membuat API Baru

1. Buatlah sebuah controller baru dengan cara klik kanan folder Controllers > Add > Controller > Add > Beri Nama “PageController”



Add New Scaffolded Item



2. Kemudian modifikasilah Controller yang dibuat agar menjadi API Controller.

```
1 using Microsoft.AspNetCore.Mvc;
2
3 namespace PRG4_M9_P1_XXX.Controllers
4 {
5     [ApiController]
6     [Route("[controller]")]
7     public class PageController : Controller
8     {
9     }
10 }
11
12
```

3. Selanjutnya buatlah sebuah endpoint baru seperti berikut.

```
1 using Microsoft.AspNetCore.Mvc;
2
3 namespace PRG4_M9_P1_XXX.Controllers
4 {
5     [ApiController]
6     [Route("[controller]")]
7     public class PageController : Controller
8     {
9         [HttpGet]
10        public String Index()
11        {
12            return "Hello World";
13        }
14    }
15 }
16
```

4. Selanjutnya jalankan projek dan cobalah api yang sudah dibuat melalui swagger, postman ataupun dipanggil di dalam web browser!

The image displays two screenshots demonstrating the API endpoint. The top screenshot shows the Swagger UI for the project 'PRG4_M9_P1_XXX'. It lists the endpoint 'GET /Page' with no parameters. The response body is shown as 'Hello World'.

The bottom screenshot shows the Postman interface. The URL is 'http://localhost:5090/Page' and the method is 'GET'. The response is '200 OK' with a body of 'Hello World'.

Below the Postman interface, a web browser address bar is shown with the URL 'localhost:5090/Page' and the text 'Hello World' displayed below it.

- **Membuat API dengan berbagai Parameter**

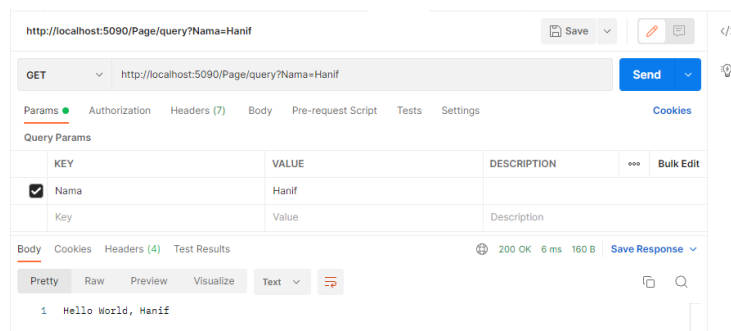
[Query Parameter]

1. Tambahkan endpoint berikut!

```
1 using Microsoft.AspNetCore.Mvc;
2
3 namespace PRG4_M9_P1_XXX.Controllers
4 {
5     [ApiController]
6     [Route("[controller]")]
7     public class PageController : Controller
8     {
9         [HttpGet]
10        public String Index()
11        {
12        }
13
14        [HttpGet]
15        [Route("query")]
16        public String Page([FromQuery] String Nama)
17        {
18            return "Hello World, " + Nama;
19        }
20    }
21 }
22
23
```

2. Panggilah API tersebut seperti berikut!

<http://localhost:5090/Page/query?Nama=Nama Anda!>



[Path Parameter]

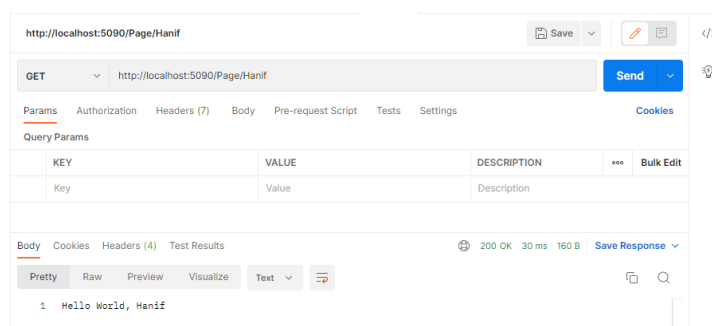
1. Tambahkan endpoint berikut!

```
[HttpGet]
[Route("query")]
public String Page([FromQuery] String Nama)

[HttpGet]
[Route("{Nama}")]
public String Page2(String Nama)
{
    return "Hello World, " + Nama;
}
```

2. Panggilah API tersebut seperti berikut!

<http://localhost:5090/Page/query/Nama Anda!>



[Body Parameter]

1. Tambahkan endpoint berikut!

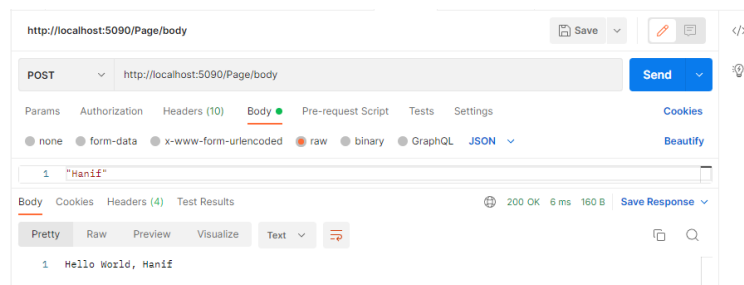
```
[HttpGet]
[Route("query")]
public String Page([FromQuery] String Nama) {...}

[HttpGet]
[Route("{Nama}")]
public String Page2(String Nama) {...}

[HttpPost]
[Route("body")]
public String Page3([FromBody] String Nama)
{
    return "Hello World, " + Nama;
}
```

2. Panggilah API tersebut seperti berikut!

[POST] http://localhost:5090/Page/body



[Header Parameter]

- Sebagai pelengkap keamanan, API akan **ditambahkan header parameter** untuk memastikan hanya pengguna terverifikasi yang bisa mengakses. Berikut contoh penggunaan **authorization bearer token**

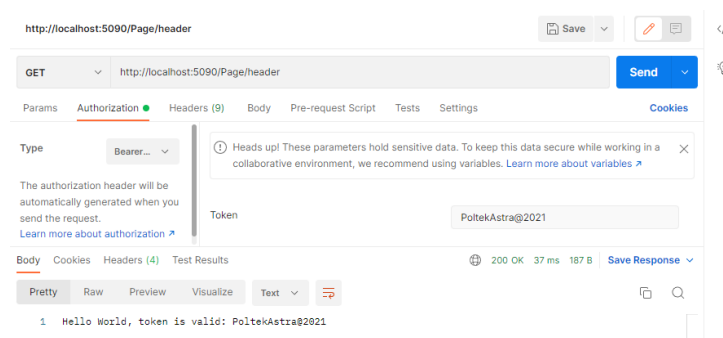
1. Tambahkan endpoint berikut!

```
[HttpPost]
[Route("body")]
public String Page3([FromBody] String Nama) {...}

[HttpGet]
[Route("header")]
public String Page4([FromHeader(Name = "Authorization")] String Authorization)
{
    if (string.IsNullOrEmpty(Authorization)) { return "Authorization header is missing"; }
    if (!Authorization.StartsWith("Bearer ")) { return "Invalid token format"; }
    var token = Authorization.Substring("Bearer ".Length).Trim();
    if (token == "PoltekAstra@2021") { return "Hello World, token is valid: " + token; }
    else { return "Invalid token"; }
}
```

2. Panggilah API tersebut seperti berikut!

http://localhost:5090/Page/header



- **Membuat API menggunakan berbagai method**

[GET]

```
[HttpGet]
[Route("get")]
0 references
public String PageMethodGet() { return "Method Get"; }
```

[POST]

```
[HttpPost]
[Route("create")]
0 references
public String PageMethodPost() { return "Method Post"; }
```

[PUT]

```
[HttpPut]
[Route("update")]
0 references
public String PageMethodPut() { return "Method Put"; }
```

[DELETE]

```
[HttpDelete]
[Route("delete")]
0 references
public String PageMethodDel() { return "Method Delete"; }
```

1. Tambahkan endpoint diatas!
2. Kemudian cobalah setiap api tersebut!

- **Best Practices Pembuatan API**

1. Gunakan HTTP Methods yang Tepat!
 - **GET** untuk mengambil data.
 - **POST** untuk membuat data baru.
 - **PUT** untuk memperbarui data yang ada.
 - **DELETE** untuk menghapus data.
 - **PATCH** untuk memperbarui sebagian data.
2. Struktur Endpoint yang Konsisten dan Deskriptif!
 - **Gunakan bentuk jamak (plural) untuk resource**, misalnya /products, /users, dll.
 - **Hindari penggunaan kata kerja pada endpoint**. Fokus pada resource yang dimaksud, seperti /products, bukan /getProducts.
3. Penggunaan Status Code yang Tepat
 - **200** untuk operasi yang berhasil.
 - **400** untuk setiap permintaan yang gagal.
 - **500** untuk setiap kesalahan internal yang terjadi pada server.

4. Penggunaan Format JSON untuk Request dan Response

- Gunakan format JSON yang konsisten dalam request dan response.
- Sebagai Contoh

```
{
  "code": "200",
  "data": { ... },
  "message": "Permintaan Berhasil"
}
```

5. Keamanan API

- Implementasikan autentikasi dan otorisasi untuk melindungi data dan operasi di API. Seperti penggunaan Authorizatoin Bearer Token, apiKey, JWT (JSON Web Token).

• Membuat API Best Practices

1. Buatlah sebuah kelas yang digunakan untuk menampung response dari API seperti berikut.

```
0 references
private Dictionary<string, Object> DtoResponse(int code, String message, object data)
{
    var response = new Dictionary<string, Object>
    {
        { "code", code },
        { "message", message },
        { "data", data }
    };
    return response;
}
```

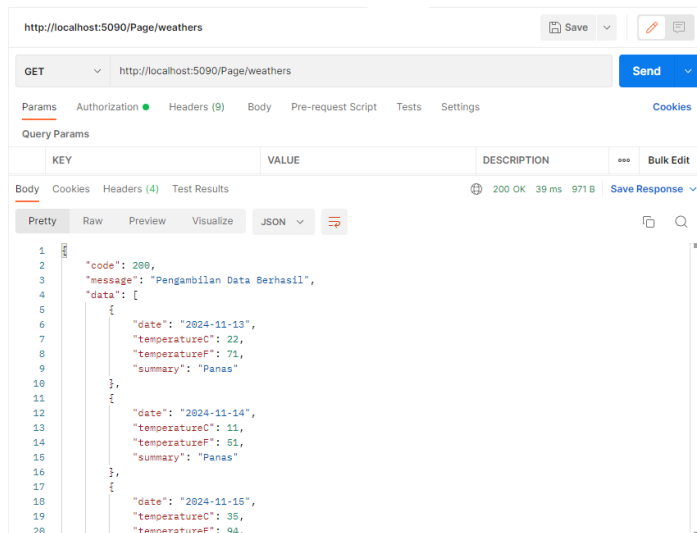
2. Selanjutnya buatlah endpoint berikut!

```
[HttpGet]
[Route("weathers")]
0 references
public IActionResult GetDataWeathers()
{
    try {
        string[] Summaries = new[]
        {
            "Cerah", "Mendung", "Berangin", "Panas", "Hujan"
        };

        var data = Enumerable.Range(1, 10).Select(index => new WeatherForecast
        {
            Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            TemperatureC = Random.Shared.Next(-20, 55),
            Summary = Summaries[Random.Shared.Next(Summaries.Length)]
        })
        .ToArray();

        return Ok(DtoResponse(200, "Pengambilan Data Berhasil", data));
    } catch (Exception ex) {
        return StatusCode(500, DtoResponse(500, "Gagal : " + ex.Message, null));
    }
}
```

3. Kemudian cobalah API tersebut!



- Tambahkan kode berikut untuk membuat error yang disengaja, sehingga kita bisa mengetahui pesan kesalahan apabila terjadi error!

```

[HttpGet]
[Route("weathers")]
public IActionResult GetDataWeathers()
{
    try {
        throw new InvalidOperationException("Terjadi kesalahan sengaja");

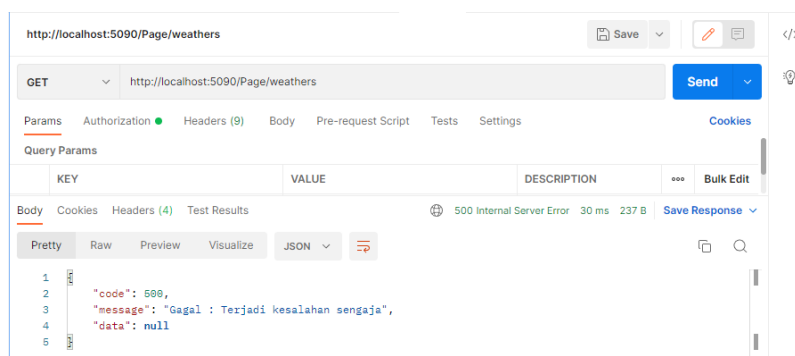
        string[] Summaries = new[]
        {
            "Cerah", "Mendung", "Berangin", "Panas", "Hujan"
        };

        var data = Enumerable.Range(1, 10).Select(index => new WeatherForecast
        {
            Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            TemperatureC = Random.Shared.Next(-20, 55),
            Summary = Summaries[Random.Shared.Next(Summaries.Length)]
        })
        .ToArray();

        return Ok(DtoResponse(200, "Pengambilan Data Berhasil", data));
    } catch (Exception ex) {
        return StatusCode(500, DtoResponse(500, "Gagal : " + ex.Message, null));
    }
}

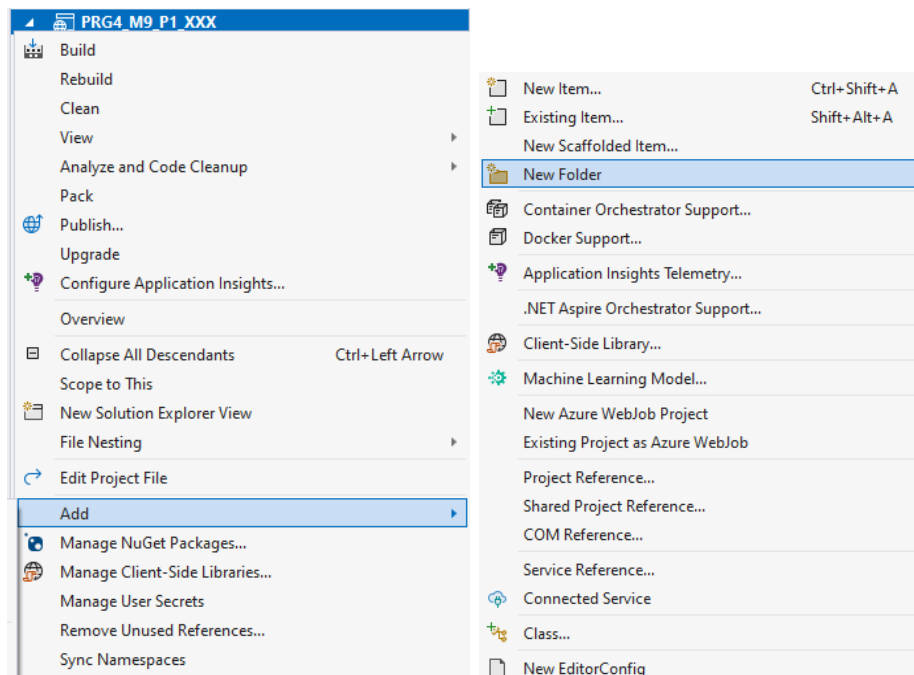
```

- Cobalah kembali API tersebut!

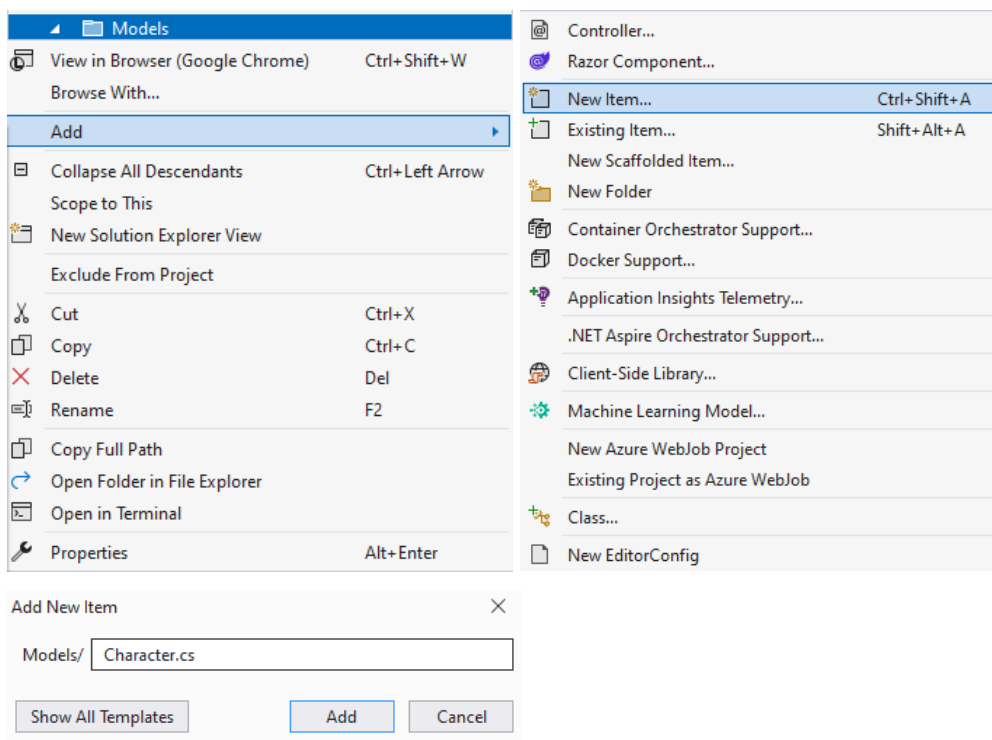


- **Membuat Model baru**

1. Buatlah folder baru dengan cara klik kanan > Add > New Folder, kemudian beri nama “Models”



2. Buatlah sebuah kelas baru dengan cara klik kanan > Add > New Item, kemudian beri nama “Character.cs”



3. Selanjutnya, tambahkan attribut pada character.cs

```

1 namespace PRG4_M9_P1_XXX.Models
2 {
3     0 references
4     public class Character
5     {
6         0 references
7         public int Id { get; set; }
8         0 references
9         public string? Name { get; set; }
10        0 references
11        public string? Description { get; set; }
12        0 references
13        public string? Origin { get; set; }
14        0 references
15        public string? Image { get; set; }
16    }
17 }

```

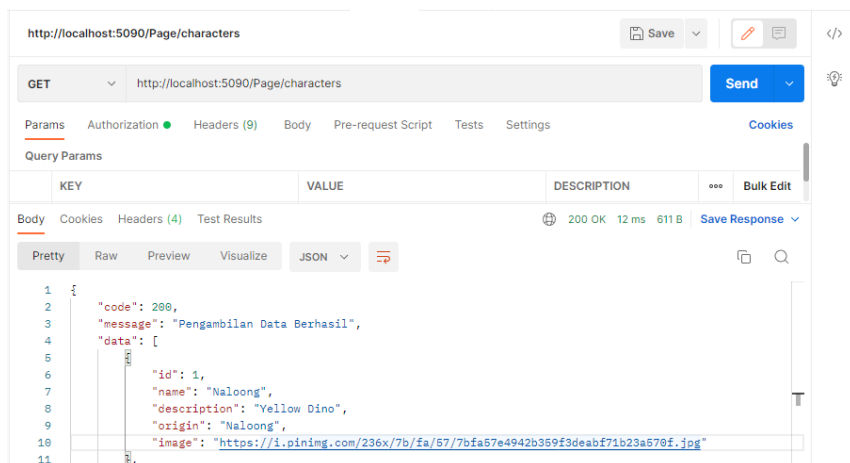
- Setelah membuat model, selanjutnya kita akan mengimplementasikan model tersebut ke dalam controller.
- Tambahkan endpoint berikut pada controller yang telah dibuat sebelumnya!

```

[HttpGet]
[Route("characters")]
0 references
public IActionResult GetDataCharacters() {
    try
    {
        List<Character> characters = new List<Character>();
        characters.Add(new Character
        {
            Id = 1,
            Name = "Naloong",
            Description = "Yellow Dino",
            Origin = "Naloong",
            Image = "https://i.pining.com/236x/7b/fa/57/7bfa57e4942b359f3deabf71b23a578f.jpg"
        });
        characters.Add(new Character
        {
            Id = 2,
            Name = "brightbill",
            Description = "burung puyuh",
            Origin = "wildrobot",
            Image = ""
        });
        characters.Add(new Character
        {
            Id = 3,
            Name = "Singa Ajaib",
            Description = "Raja Hutan",
            Origin = "Paddle Pop",
            Image = "https://paddle-pop-adventures.fandom.com/wiki/Paddle_Pop?file=Max.png"
        });
        return Ok(DtoResponse(200, "Pengambilan Data Berhasil", characters));
    }
    catch (Exception ex)
    {
        return StatusCode(500, DtoResponse(500, "Gagal : " + ex.Message, null));
    }
}

```

- Selanjutnya cobalah API tersebut!



The screenshot shows a web browser with the URL `http://localhost:5090/Page/characters`. The response is a JSON object with the following structure:

```

{
  "code": 200,
  "message": "Pengambilan Data Berhasil",
  "data": [
    {
      "id": 1,
      "name": "Naloong",
      "description": "Yellow Dino",
      "origin": "Naloong",
      "image": "https://i.pining.com/236x/7b/fa/57/7bfa57e4942b359f3deabf71b23a578f.jpg"
    },
    {
      "id": 2,
      "name": "brightbill",
      "description": "burung puyuh",
      "origin": "wildrobot",
      "image": ""
    },
    {
      "id": 3,
      "name": "Singa Ajaib",
      "description": "Raja Hutan",
      "origin": "Paddle Pop",
      "image": "https://paddle-pop-adventures.fandom.com/wiki/Paddle_Pop?file=Max.png"
    }
  ]
}

```

6. Latihan

- Buatlah API dengan ketentuan berikut
 1. Menggunakan Best Practices
 2. Menggunakan Model dengan tema tertentu
 3. Menggunakan method GET, POST, PUT, DELETE
 4. Menggunakan berbagai parameter